With this, we've completed the tools review. Now, let's discuss what to consider when adopting tooling in the space.

9.3 Tooling considerations

We have examined continuous profiling and, more generally, developer productivity tooling in this chapter, and you may by now be excited to jump into the deep end. Before you get too far, I'd like to share a few thoughts and words of caution in the following subsections. While I'm personally super bullish and supportive, we have to recognize sharp edges and set expectations correctly.

Developer observability is a relatively young field, with many of the contenders having been on the market for less than a year. The following is a discussion of challenges that CP tooling faces specifically.

9.3.1 Symbolization

By *symbolization*, we mean the process of translating machine addresses to human-readable (symbolic) names. For example, if you look at a profile and it contains stack frame labels, such as `0x8c6fff910`, you would not really know what that memory address signifies. On the other hand, if you were to see something like `/some/path/file.go:42` or `calculateSummary()`, that would be meaningful.

The challenge now is to determine the location of symbolic information. For compiled languages, such as C/C++ or Go, the symbol table (containing symbols used) is usually part of the binary, and for interpreted languages, including Python, Ruby, or JavaScript, the symbols are typically part of the program.

For further reading on this topic, see also these blog posts:

- "Fantastic Symbols and Where to Find Them" by Kemal Akkoyun (http://mng.bz/QPB6)
- "BPF binaries: BTF, CO-RE, and the Future of BPF Perf tools" by Brendan Gregg (http://mng.bz/XNR9)
- "BPF CO-RE (Compile Once – Run Everywhere)" by Andrii Nakryiko (https://nakryiko.com/posts/bpf-portability-and-co-re/)

Once we have the symbols, we're not done yet; we still need to store the profiles. But how can we do that in an efficient and budget-friendly manner?

9.3.2 Storing profiles

In-memory storage of profiles is sufficient for certain use cases when you are examining what is happening in the moment. However, in cases when you want to compare longer time ranges or different versions of your service, you need to be able to deal with the volume of the profiles and store them somewhere in a persistent manner.

Two common techniques found in continuous profiling tools are

- *To efficiently store and query profiles, columnar storage is used (see also chapter 5, where we discussed the design and properties of columnar data stores).* For example, in the